In [ ]:
```
# Input functions continuation
```

In [1]:
```
exp=input('Enter a value')
print(exp)
```

```
2+3-4
```

In [2]:
```
exp=int(input('Enter a value')) #expressions work only with evaluate called eval
print(exp)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[2], line 1
----> 1 exp=int(input(               )) #expressions work only with evaluate called
eval
      2 print(exp)

ValueError: invalid literal for int() with base 10: '2+3-4'
```

In [3]:
```
exp=eval(input('Enter a value')) # thus its working
print(exp)
```

```
-1
```

List Data Structures

it always defines with square braces

In [4]:
```
l = []
l
```

Out[4]:
```
[]
```

In [5]:
```
type(l)
```

Out[5]:
```
list
```

In [6]:
```
type(list(l))
```

Out[6]:
```
list
```

In [7]:
```
l.append(20)     #adding the value at end of list is done using append
l.append(50)
l.append(70)     # you can append many number of times
l
```

Out[7]:
```
[20, 50, 70]
```

In [8]:
```
len(l)
```

Out[8]: 3

In [9]:
```
l.append(20)   #it allows duplicates as well
l
```

Out[9]: [20, 50, 70, 20]

In [10]:
```
l1 = [20, 3.4, 3+2j, True,'hello', [1,4,5]]  #heterogenoue i.e all data types pf va
l1
```

Out[10]: [20, 3.4, (3+2j), True, 'hello', [1, 4, 5]]

In [11]:
```
len(l1)
```

Out[11]: 6

In [12]:
```
len(l)
```

Out[12]: 4

In [14]:
```
print(l)
print(l1)
```

```
[20, 50, 70, 20]
[20, 3.4, (3+2j), True, 'hello', [1, 4, 5]]
```

In [15]:
```
print(id(l))
print(id(l1))   #memory location address is called as ID
```

```
2685132551616
2685132537088
```

In [18]:
```
l2 = l.copy()
l2
```

Out[18]: [20, 50, 70, 20]

In [19]:
```
len(l2)
```

Out[19]: 4

In [20]:
```
l2.clear()  # it clears the l2 data and length becomes zero
```

In [21]:
```
len(l2)
```

Out[21]: 0

In [22]:
```
del l2
#the del will delete the l2 since it doesnt have any values in it
#where it automaticlly goes to garbage collector
```

In [23]:
```
l2
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[23], line 1
----> 1 l2

NameError: name 'l2' is not defined
```

In [24]:
```
l.count(20)  # this is used to count how many times 20 is available in l
```

Out[24]:
```
2
```

In [25]:
```
l.count(100)
```

Out[25]:
```
0
```

In [26]:
```
print(l)
print(l1)
```

```
[20, 50, 70, 20]
[20, 3.4, (3+2j), True, 'hello', [1, 4, 5]]
```

In [27]:
```
a = [1,2,3] #extend is used to extend the variable values using the other variables
b = [4,5,6]
a.extend(b)
```

In [29]:
```
a
```

Out[29]:
```
[1, 2, 3, 4, 5, 6]
```

In [28]:
```
l2 = []
l2
```

Out[28]:
```
[]
```

In [30]:
```
l2.extend(l)  #the values of l will be given to l2
l2
```

Out[30]:
```
[20, 50, 70, 20]
```

In [31]:
```
l2.extend(l1)
l2
```

Out[31]:
```
[20, 50, 70, 20, 20, 3.4, (3+2j), True, 'hello', [1, 4, 5]]
```

In [84]:
```
a = [1,2,3]
b = [4,5,6]

a.extend(b)
a
```

Out[84]:
```
[1, 2, 3, 4, 5, 6]
```

In [32]:
```
l[0]
```

Out[32]: 20

In [33]:
```
l[0] = 100 #list is mutable(changing the list value by its index position)
l
```

Out[33]: [100, 50, 70, 20]

In [34]:
```
l[1] = 500
l
```

Out[34]: [100, 500, 70, 20]

In [35]:
```
l.index(500)
```

Out[35]: 1

In [36]:
```
l.index(20) #it returns the first index position value but if we have duplicate val
```

Out[36]: 3

In [37]:
```
l.insert(2,33) #using insert we can insert values btween the values selected (index
l
```

Out[37]: [100, 500, 33, 70, 20]

In [38]:
```
l1
```

Out[38]: [20, 3.4, (3+2j), True, 'hello', [1, 4, 5]]

In [39]:
```
l1.pop()  #it will remove last element
```

Out[39]: [1, 4, 5]

In [40]:
```
l1
```

Out[40]: [20, 3.4, (3+2j), True, 'hello']

In [41]:
```
l1.pop(3)  #if we give argument..it deletes the 3rd position value
```

Out[41]: True

In [42]:
```
l1
```

Out[42]: [20, 3.4, (3+2j), 'hello']

In [43]:
```
l2
```

Out[43]: [20, 50, 70, 20, 20, 3.4, (3+2j), True, 'hello', [1, 4, 5]]

In [44]:
```
l2.remove(20)
l2
```

Out[44]: [50, 70, 20, 20, 3.4, (3+2j), True, 'hello', [1, 4, 5]]

In [45]:
```
l.reverse()  #it just mentions the reverse orderl
l
```

Out[45]: [20, 70, 33, 500, 100]

In [46]:
```
l.sort()
l
```

Out[46]: [20, 33, 70, 100, 500]

In [47]:
```
l.sort(reverse=True)  #it reverse funtion is added and is TRUE,the values are sorte
l
```

Out[47]: [500, 100, 70, 33, 20]

In [48]:
```
l.sort(reverse=False)  #asc order
l
```

Out[48]: [20, 33, 70, 100, 500]

In [50]:
```
l3= [10,20,30,40]
l3[2] = [13]
l3
```

Out[50]: [10, 20, [13], 40]

In [51]:
```
l4 = [12,15,67,100,29]
l4
```

Out[51]: [12, 15, 67, 100, 29]

In [53]:
```
l4.sort()
l4
```

Out[53]: [12, 15, 29, 67, 100]

In [52]:
```
l4.sort(reverse=False) #parameter tuning is nothing but if u define rev=false and n
l4
```

Out[52]: [12, 15, 29, 67, 100]

In [ ]:
```
l5 = [c, 3.5,'ch', True]
```

In [54]:
```
l5.sort()
l5
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[54], line 1
----> 1 l5.sort()
      2 l5

NameError: name 'l5' is not defined
```

In [ ]:
```
# list indexing and slicing
```

In [60]:
```
l6 = [30,47,96,100]
l6
```

Out[60]:
```
[30, 47, 96, 100]
```

In [56]:
```
l1
```

Out[56]:
```
[20, 3.4, (3+2j), 'hello']
```

In [57]:
```
l1[:]
```

Out[57]:
```
[20, 3.4, (3+2j), 'hello']
```

In [58]:
```
l1[::-2]  #it deducts -2 value from back
```

Out[58]:
```
['hello', 3.4]
```

In [66]:
```
l1[0:15:2]   # 0 is index  15 is n times 2 is every slcing time
```

Out[66]:
```
[20, (3+2j)]
```

tuple or tuple

In [ ]:
```
#tuple are defines as open braces and closed braces
```

In [68]:
```
t = ()
t1 = (10,30,20,40)
t1
```

Out[68]:
```
(10, 30, 20, 40)
```

In [71]:
```
t1.count(10)
#it has only 2 functions COUNT and INDEX
```

Out[71]:
```
1
```

In [72]:
```
t1.count(100)
```

Out[72]:
```
0
```

In [73]:
```
t1.index(30)
```

Out[73]:
```
1
```

In [74]:
```
t1[2] = 15  #TUPLES ARE NOT MUTABLE AND LIST IS MUTABLE
            # LIKE AADHAAR CARD TO MAKE UR ACCOUNT SECURED
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[74], line 1
----> 1 t1[2] = 15

TypeError: 'tuple' object does not support item assignment
```

In [76]:
```
t1
```

Out[76]:
```
(10, 30, 20, 40)
```

In [77]:
```
t1 * 3  #when tuples are multipled, the elements are repeated 3 times
```

Out[77]:
```
(10, 30, 20, 40, 10, 30, 20, 40, 10, 30, 20, 40)
```

SET

It is mentioned using curly braces {}

In [78]:
```
s = {}
s        #the empty braces are taken as dict
```

Out[78]:
```
{}
```

In [79]:
```
type(s)
```

Out[79]:
```
dict
```

In [80]:
```
s1 = set()
s1
```

Out[80]:
```
set()
```

In [90]:
```
s1 = {10,40,57,49,50}   #once u write the values inside set, it sorts them by defau
s1
```

Out[90]:
```
{10, 40, 49, 50, 57}
```

In [82]:
```
type(s1)
```

Out[82]:
```
set
```

In [85]:
```
s1.add(18)  # it directly adds in set in sorted manner
s1
```

Out[85]:
```
{10, 18, 40, 49, 50, 57, 100}
```

In [86]:
```
s2 = {5,5.5,'nit', 2+3j,True}  # with different data types it will add randomly any
s2
```

Out[86]:
```
{(2+3j), 5, 5.5, True, 'nit'}
```

In [99]:
```
print(s1)
print(s2)
```

```
{49, 50, 40, 57, 10}
{True, 'nit', 5.5, 5, (2+3j)}
```

In [109…
```
s3 = s2.copy()
s3
```

Out[109…
```
{(2+3j), 5, 5.5, True, 'nit'}
```

In [101…
```
s3[0]   #INDEXING & SLICING IS NOT ALLOWED IN SETS & ALLOWED IN LIST,TUPLES ,DICT
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[101], line 1
----> 1 s3[0]   #INDEXING & SLICING IS NOT ALLOWED IN SETS & ALLOWED IN LIST,TUPLES
,DICT

TypeError: 'set' object is not subscriptable
```

In [102…
```
s3
```

Out[102…
```
{(2+3j), 5, 5.5, True, 'nit'}
```

In [103…
```
s3.pop()  #it deletes one element randomly
```

Out[103…
```
True
```

In [104…
```
s3
```

Out[104…
```
{(2+3j), 5, 5.5, 'nit'}
```

In [105…
```
s3.pop(2)  #indexing is not allowed here as well to delete
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[105], line 1
----> 1 s3.pop(2)

TypeError: set.pop() takes no arguments (1 given)
```

In [106…
```
s3.clear()
```

In [110…
```
s3
```

Out[110…
```
{(2+3j), 5, 5.5, True, 'nit'}
```

In [111…
```
s3.discard(2+3j)
```

In [112…
```
s3
```

Out[112…
```
{5, 5.5, True, 'nit'}
```

In [114…
```
s3.discard(500)  # In discard, if element is not available it doesnt throw error bu
```

Out[114…
```
{5, 5.5, True, 'nit'}
```

In [115…
```
s3.remove(500) # remove throws an error
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[115], line 1
----> 1 s3.remove(500)

KeyError: 500
```

In [117…
```
s3.add(5)  # In sets duplicates are not allowed
s3
```

Out[117…
```
{5, 5.5, True, 'nit'}
```

SET OPERATION

In [ ]:
```
We have different types of operations
UNION , INTERSECTION, DIFFERENCE, SYMMETRIC DIFFERENCE, INTERSECTION UPDATE, SUPERS
```

In [1]:
```
s5 = {1,2,3,4,5}
s6 = {4,7,8,9,10}
s7 = {11,12,13,14}
```

In [2]:
```
s5.union(s6)    #duplicates are not allowed in sets
```

Out[2]:
```
{1, 2, 3, 4, 5, 7, 8, 9, 10}
```

In [3]:
```
s5.union(s6,s7)
```

Out[3]:
```
{1, 2, 3, 4, 5, 7, 8, 9, 10, 11, 12, 13, 14}
```

In [4]:
```
s6|s7
```

Out[4]:
```
{4, 7, 8, 9, 10, 11, 12, 13, 14}
```

In [5]:
```
s5|s6|s7
```

Out[5]: {1, 2, 3, 4, 5, 7, 8, 9, 10, 11, 12, 13, 14}

In [6]:
```
print(s7)
print(s5)
print(s6)
```

```
{11, 12, 13, 14}
{1, 2, 3, 4, 5}
{4, 7, 8, 9, 10}
```

In [ ]:
```
#intersections
```

In [9]:
```
s5.intersection(s6)
```

Out[9]: {4}

In [10]:
```
s6.intersection(s7)
```

Out[10]: set()

In [11]:
```
s7 & s6
```

Out[11]: set()

In [27]:
```
s9 = {1,3,5,7,9,10,11}
s10 = {1,2,4,5,8,9}
s11 = {2,3,4,9,10}
s9
s10
s11
```

Out[27]: {2, 3, 4, 9, 10}

In [28]:
```
s11 - s10  #difference gives the values which are in s11 and non common elements
```

Out[28]: {3, 10}

In [29]:
```
s9 - s10 # different values from s9 are only executed in o/p
```

Out[29]: {3, 7, 10, 11}

In [31]:
```
s9.difference(s10)
```

Out[31]: {3, 7, 10, 11}

In [43]:
```
s10.symmetric_difference(s11)   # it returns values which are not common in both se
```

Out[43]: {1, 3, 5, 8, 10}

In [45]:
```
s9 ^ s10 #(symbol of symmetric difference)
```

Out[45]: {1, 3, 5, 7, 9, 10, 11}

In [41]:
```
s9 = {1,3,5,7,9,10,11}
s10 = {1,2,4,5,8,9}
s11 = {2,3,4,9,10}
```

In [48]:
```
s9.symmetric_difference_update(s10)  #it returns values which are not common and al
print(s9)
print(s10)
```

```
{3, 5, 7, 1, 10, 11, 9}
{1, 2, 4, 5, 8, 9}
```

superset , subset, disjoint

In [49]:
```
a = {1,2,3,4,5,6,7,8,9}
b = {3,4,5,6,7,8}
c = {10,11,12}
```

In [50]:
```
a.issuperset(b)
```

Out[50]:
```
True
```

In [51]:
```
b.issubset(a)
```

Out[51]:
```
True
```

In [52]:
```
a.issubset(b)
```

Out[52]:
```
False
```

In [53]:
```
a.isdisjoint(b)  #disjoint means it will not ahve common elements and returns True.
```

Out[53]:
```
False
```

In [54]:
```
a.isdisjoint(c)
```

Out[54]:
```
True
```

In [ ]:

In [ ]: